Virtual Server Monitoring Pipeline

By: **Sohil S Philip | Data Engineer | sohilphilip137@gmail.com**

Role: **Data Engineering Case Study Submission**

Date: **March 2026**

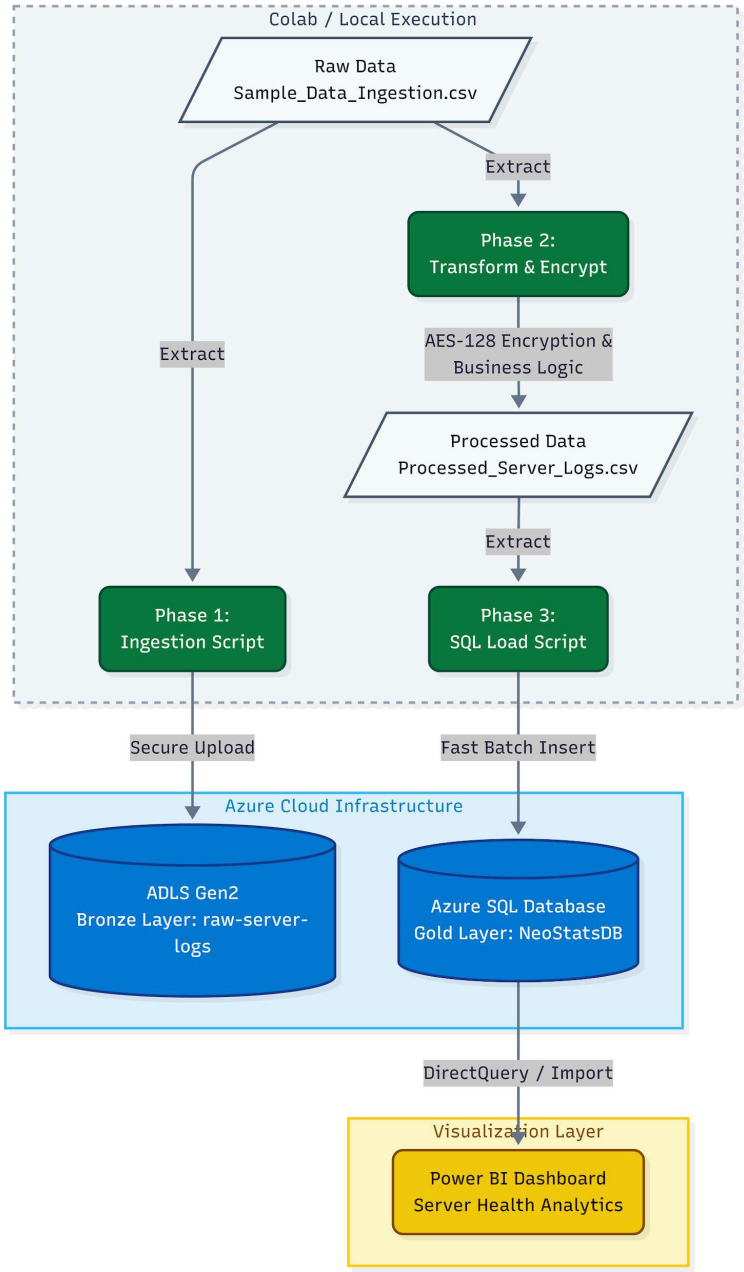
1. Executive Summary

XYZ Corporation faces challenges in monitoring server performance and identifying resource bottlenecks across its distributed cloud infrastructure. This project provides a fully automated, end-to-end Data Engineering pipeline. It securely ingests raw server logs, applies AES-128 encryption to sensitive Personally Identifiable Information (PII), enriches the data with business logic, and lands it in a cloud data warehouse for real-time visualization in Power BI.

2. High-Level Architecture & Workflow

The solution is built on a modern cloud data stack using Microsoft Azure and Python. The workflow is divided into three primary zones:

1. **Layer1 (Raw Storage):** Raw CSV logs are ingested into Azure Data Lake Storage (ADLS) Gen2.
2. **Layer2 (Processing & Security):** Python (via Pandas and Cryptography) standardizes metrics, categorizes server sizes, and encrypts PII.
3. **Layer3 (Serving):** The enriched, secured dataset is loaded into Azure SQL Database, which acts as the direct source for the Power BI dashboard.



3. Cloud Infrastructure Setup

To run this pipeline, the following Azure resources must be configured. Follow these step-by-step instructions to provision the environment.

1) **Step 1:** Create a Resource Group

   A Resource Group is a logical folder that holds all the different Azure services you will create for this specific project, making them easy to manage.

   a) Log In: Open your web browser and go to portal.azure.com.
   b) Find Resource Groups: On the main dashboard, search for **"Resource groups"** in the top search bar and click on it.
   c) Create: Click the "+ Create" button.
   d) Configure: * Subscription: Leave the default.
   e) Resource group: Name it **NeoStats-DataPipeline-RG.**
   f) Region: Select a region geographically closest to you.
   g) Deploy: Click "Review + create", and then "Create".

2) **Step 2:** Azure Data Lake Storage (ADLS) Gen2
   a) This will serve as the landing zone for the raw CSV files.
   b) Create Storage Account: Search for **"Storage accounts"** in the Azure Portal and click Create. Select your **NeoStats-DataPipeline-RG.**
   c) Name it: Give it a globally unique name (e.g., neostatsdatalake2026). Select your region.
   d) Enable Gen2: Go to the Advanced tab. Under Data Lake Storage Gen2, check the box for **"Enable hierarchical namespace".**
   e) Deploy: Click Review + create, then Create.
   f) Create Container: Once deployed, go to the resource. On the left menu under "Data storage", click Containers. Create a new container named **raw-server-logs.**
   g) Get Credentials: On the left menu under "**Security + networking**", click Access keys. Click "**Show**" next to key1 Connection string, and copy it. Save this to your clipboard for Step 4.

3) **Step 3:** Azure SQL Database

   This will serve as the **serving layer** for the Power BI dashboard.

   a) Create Database: Search for "**SQL databases**" in the Azure Portal and click Create. Select your **NeoStats-DataPipeline-RG**.
   b) Configure Database: Enter the Database name: **NeoStatsDB.**
   c) Create Server: Under Server, click Create new.
      i) Enter a unique server name (e.g., neostats-sql-server).
      ii) Choose Use SQL authentication.

iii) Enter an admin username and a strong password. Note these down.

d) Compute Tier: Under Compute + storage, click "Configure database" and select the Basic or Serverless tier to minimize costs.

e) Configure Firewall: Go to the Networking tab. Under Firewall rules:

i) Select Add current client IPv4 address.

ii) Important for Google Colab: You must add a rule to allow Colab's cloud servers to connect. Add a rule named Allow_Colab with **Start IP 0.0.0.0** and End IP **255.255.255.255.**

f) Deploy: Click Review + create, then Create.

g) Get Details: Once deployed, go to the database Overview page and copy your Server Name (e.g., your-server.database.windows.net).

4) **Step 4:** Google Colab Secrets Configuration

To ensure enterprise security, this project does not use hardcoded credentials. You must configure the built-in Secrets manager in the provided Google Colab notebook.

a) Open the provided NeoStats_Data_Pipeline.ipynb Google Colab link.

i) Click the Key icon **(Secrets)** on the left-hand sidebar.

ii) Click Add new secret and create the following 5 entries using the credentials gathered in Steps 2 and 3:

(1) **AZURE_STORAGE_CONNECTION_STRING**

(2) **SQL_SERVER (e.g., server.database.windows.net)**

(3) **SQL_DATABASE (e.g., NeoStatsDB)**

(4) **SQL_USERNAME**

(5) **SQL_PASSWORD**

iii) **Critical: Ensure the "Notebook access" toggle next to every secret is switched ON so the pipeline can securely read them.**

4. ETL Pipeline Implementation (Python)

The pipeline was developed as a modular script (consolidated into a Jupyter Notebook for evaluation) utilizing pandas, sqlalchemy, azure-storage-file-datalake, and cryptography.

4.1 Data Ingestion (upload_to_adls)

The script automatically detects the raw log file (eg.Sample_Data_Ingestion.csv) and uses the Azure SDK to programmatically upload it to the **ADLS Gen2 raw-server-logs container.**

4.2 Data Transformation & Enrichment (transform_data)

- Data Cleaning: The **Log_Timestamp** is standardized to a uniform Pandas datetime format, explicitly handling Day-First European formatting **(format='mixed', dayfirst=True).**

- Metric Calculation: A new column, **Total_Network_Traffic (MB/s),** is calculated by summing **Network_Traffic_In and Network_Traffic_Out.**
- Business Logic (Instance Sizing): Servers are categorized to identify bottlenecks using the industry-standard **30/70 rule**:
- **< 30% Memory Usage = Small (Underutilized)**
- **30% - 70% Memory Usage = Medium (Optimal)**
- **> 70% Memory Usage = Large (Heavy Load/Bottleneck risk)**

4.3 Advanced Security (PII Encryption)

- A strict security protocol is enforced before the data leaves the processing layer. An **AES-128 Fernet cipher** is generated per session. Sensitive columns **(IP_Address, Admin_Name, Admin_Email, Admin_Phone)** are converted to bytes, encrypted, and stored as secure hash strings in the database.

4.4 Data Loading (load_to_sql)

- The processed dataframe is pushed to the Azure SQL **ServerPerformanceMetrics** table using **SQLAlchemy** and the **ODBC Driver 18 for SQL Server**. The **fast_executemany=True** parameter is utilized to optimize batch insert performance.

5. Data Schemas

5.1 Raw Input Schema (Source)

Column Name	Data Type	Description
Server_ID	String	Unique identifier for the virtual server.
Hostname	String	Human-readable network name.
IP_Address	String	Plain-text server IP address (PII).
OS_Type	String	Operating System version.
Server_Location	String	Geographical data center location.

CPU_Utilization (%)	Decimal	Raw CPU usage percentage.
Memory_Usage (%)	Decimal	Raw RAM usage percentage.
Disk_IO (%)	Decimal	Raw Disk I/O percentage.
Network_Traffic_In	Decimal	Raw inbound traffic (MB/s).
Network_Traffic_Out	Decimal	Raw outbound traffic (MB/s).
Uptime (Hours)	Decimal	Total active hours.
Downtime (Hours)	Decimal	Total inactive hours.
Admin_Name	String	Plain-text administrator name (PII).
Admin_Email	String	Plain-text administrator email (PII).
Admin_Phone	String	Plain-text administrator phone (PII).
Log_Timestamp	String	Original log timestamp string.

5.2 Processed Schema (Azure SQL Target)

Column Name	Data Type	Type
Server_ID	String	Metadata
Hostname	String	Metadata
IP_Address	**Encrypted**	Secured
OS_Type	String	Category
Server_Location	String	Category
CPU_Utilization (%)	Decimal	Metric
Memory_Usage (%)	Decimal	Metric
Disk_IO (%)	Decimal	Metric
Uptime (Hours)	Decimal	Reliability
Downtime (Hours)	Decimal	Reliability
Admin_Name	**Encrypted**	Secured
Admin_Email	**Encrypted**	Secured
Admin_Phone	**Encrypted**	Secured
Log_Timestamp	DateTime	Time
Instance_Size	String	Enriched
Total_Network_Traffic	Decimal	Calculated

6. Power BI Visualization & Analytics

The final phase of the project delivers a highly interactive Power BI dashboard to monitor the processed server metrics.

6.1 Data Connection Strategy

To support real-time monitoring of server health, the dashboard connects to the Azure SQL Database.

- DirectQuery / Import: The dashboard is optimized to pull live data from the ServerPerformanceMetrics table.
- Automatic Page Refresh (APR): (If using DirectQuery) The dashboard is configured to automatically refresh every 5 seconds, ensuring that as soon as the Python ETL pipeline completes a load, the visuals update without user intervention.

6.2 Key Performance Indicators (DAX)

Custom DAX measures were created to aggregate overall system health:

Avg CPU Utilization =AVERAGE(ServerPerformanceMetrics[CPU_Utilization (%)])
Avg CPU Utilization = AVERAGE(ServerPerformanceMetrics[CPU_Utilization (%)])
Avg CPU Utilization = AVERAGE(ServerPerformanceMetrics[CPU_Utilization (%)])

6.3 Dashboard Layout & Insights

The interactive report features:

1. **Resource Overview:** A Donut Chart visualizing the Instance_Size distribution. Due to the 30/70 business logic implemented in Python, the dashboard clearly highlights the distribution of heavily loaded vs. underutilized servers.
2. **Performance Trends:** A Line Chart tracking Avg CPU and Avg Memory over Log_Timestamp to spot temporal spikes.
3. **Interactivity:** Slicers for OS_Type and Server_Location allow administrators to drill down into specific regional or system-level bottlenecks.

NeoStats Server Health Monitoring

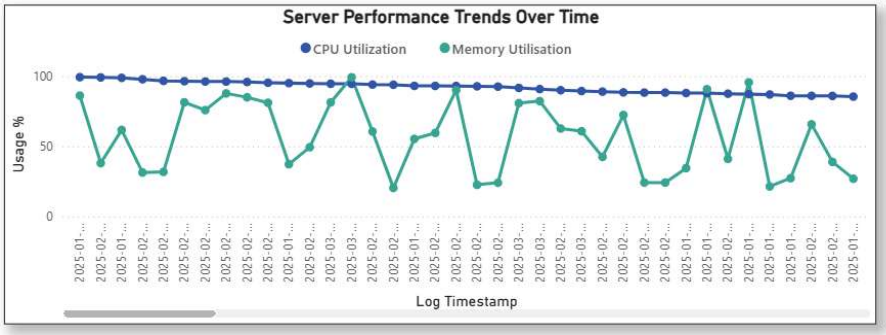
57.30%
Avg CPU Utilization

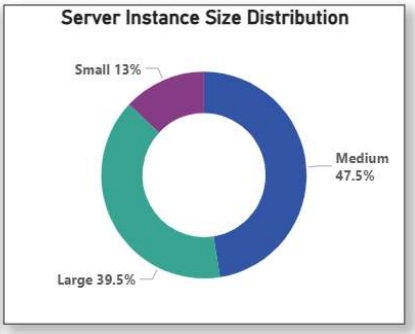
60.14%
Avg Memory Usage

31.57K
Total Network Traffic

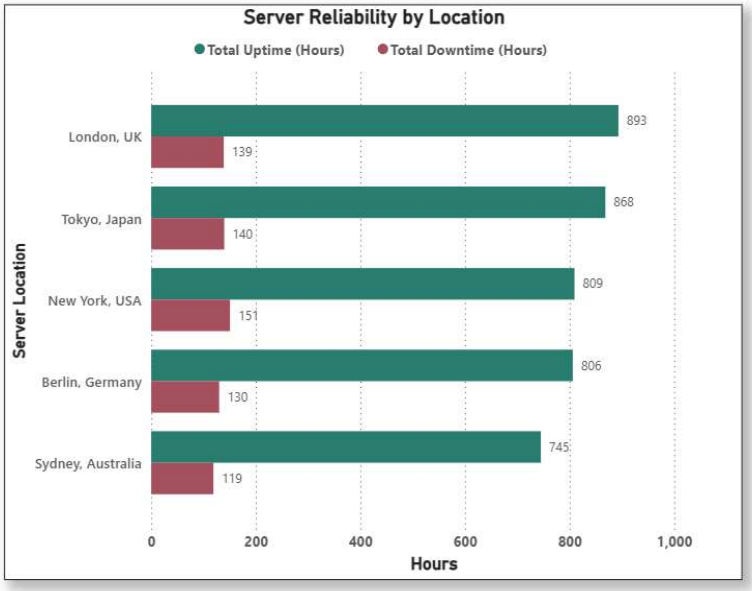
OS Type
- CentOS 8
- Red Hat Enterprise Linux
- Ubuntu 20.04
- Windows Server 2019
- Windows Server 2022

Server Location
- Berlin, Germany
- London, UK
- New York, USA
- Sydney, Australia
- Tokyo, Japan





NeoStats Server Health Monitoring



OS Type
- CentOS 8
- Red Hat Enterprise Linux
- Ubuntu 20.04
- Windows Server 2019
- Windows Server 2022

Server Location
- Berlin, Germany
- London, UK
- New York, USA
- Sydney, Australia
- Tokyo, Japan

OS Type	Avg Memory Usage	Avg CPU Usage
Windows Server 2022	65.94%	55.99%
Windows Server 2019	61.79%	59.90%
Red Hat Enterprise Linux	59.01%	54.05%
CentOS 8	58.50%	56.59%
Ubuntu 20.04	56.78%	60.62%
Total	**60.14%**	**57.30%**